

INF3500

Conception et réalisation de systèmes numériques

Laboratoire 3

À remettre: Marc-Antoine Manningham

Soumis par:
Hua, Ivan - 2400805
William Nguyễn Lê- 2393842

Groupe: 2

Date: 03-11-2025

Partie 1 : Conception du module de calcul de la racine carré et modélisation en VHDL

Pour valider le bon fonctionnement de `racine_carree.vhd`, nous avons simulé plusieurs cas, voici quelques uns en forçant des valeurs.

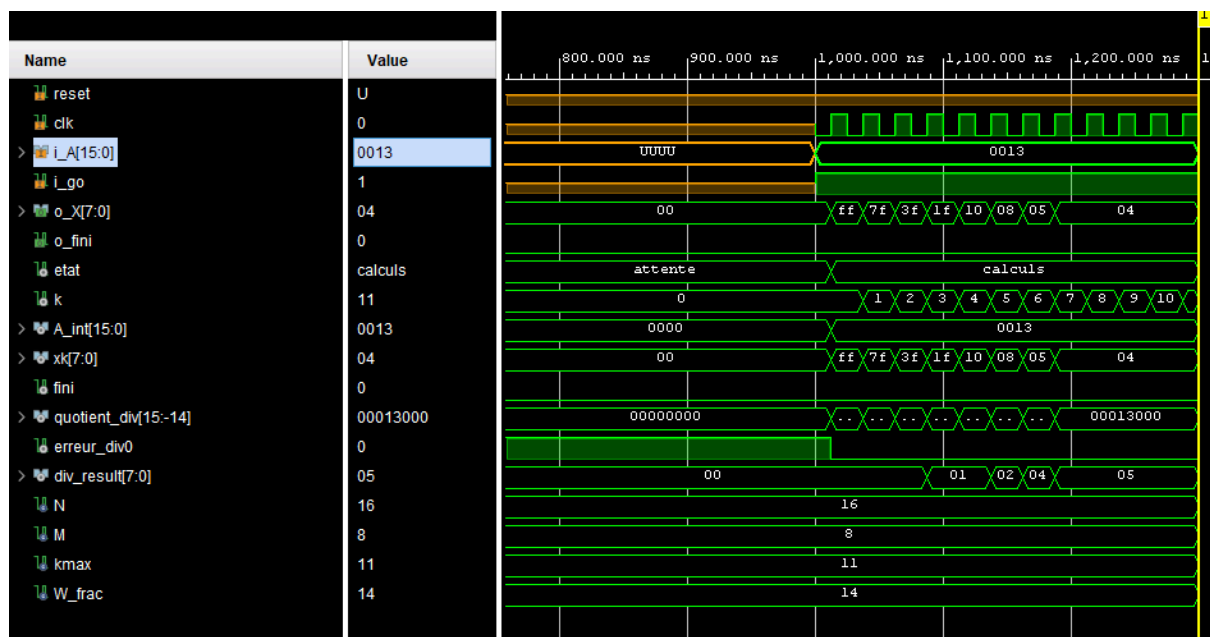


Figure 1. simulation1 du module `racine_carree_tb.vhd`

Comme vous pouvez le constater à la figure 1, nous avons testé la valeur de A prenant 0x13 qui est 19 en décimal, la racine carré la plus proche est 16 donc cela devrait nous donner 4, ce qui est la valeur de la sortie `o_X` à la fins des 10 itérations comme attendu.

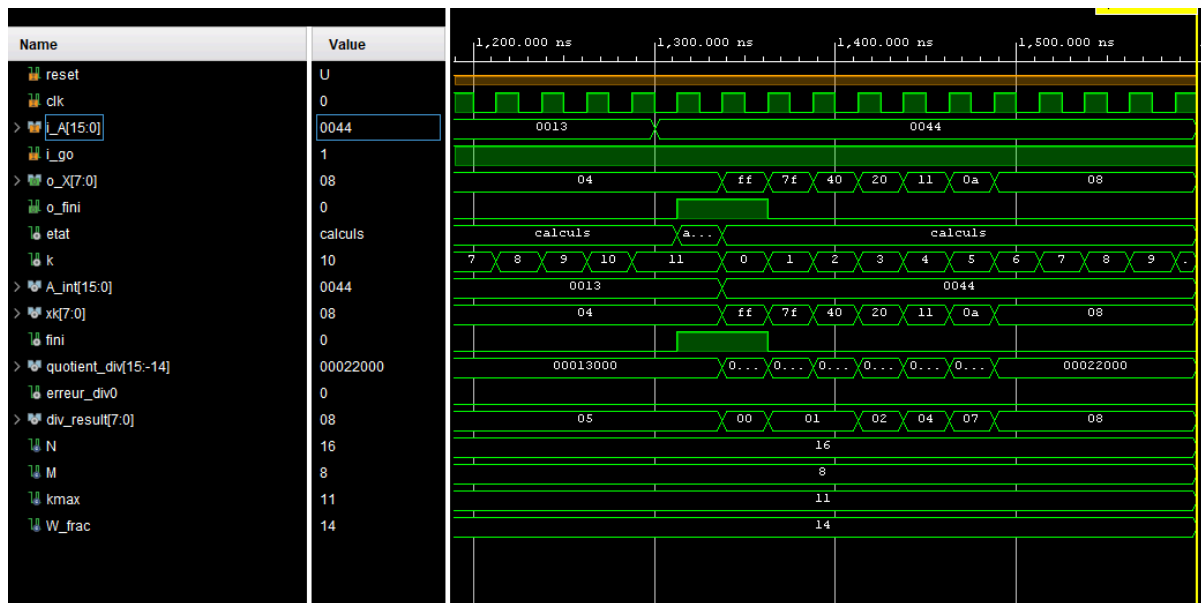


Figure 2. simulation2 du module racine_carree_tb.vhd

Comme on peut le voir à la figure 2, nous avons testé la valeur de 0x44 qui est 68 en décimal, donc encore une fois la racine carree la plus proche est 8, ce qui est la valeur de la sortie o_X attendu qu'on a obtenu.

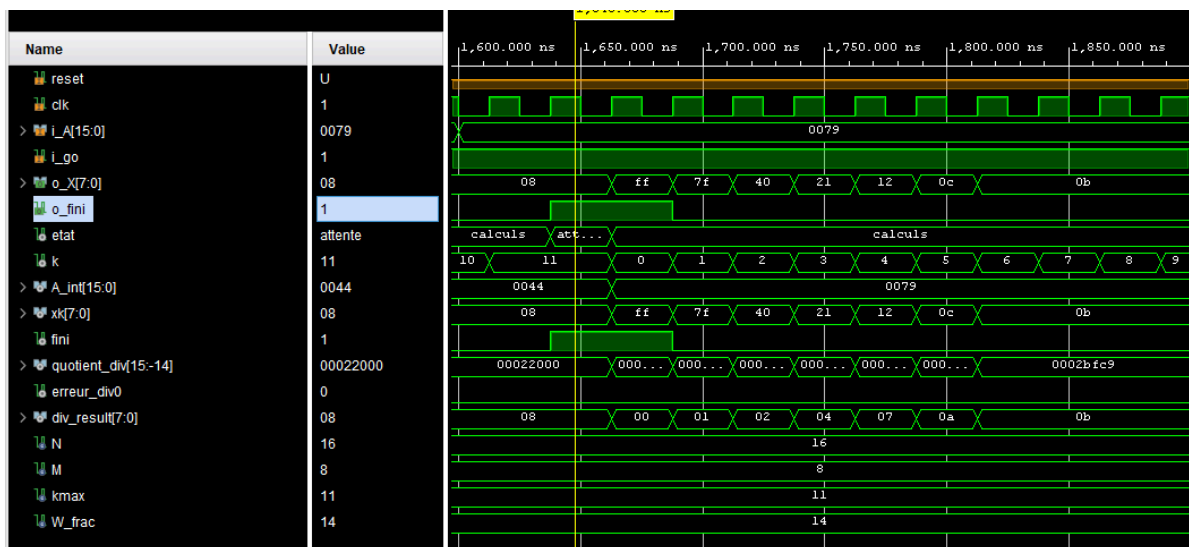


Figure 3. simulation3 du module racine_carree_tb.vhd

Comme on peut le voir à la figure 3, nous avons testé la valeur de 0x79 qui est 121 en décimal, donc encore une fois la racine carree la plus proche est 11, ce qui est la valeur de la sortie o_X attendu qu'on a obtenu 0x0b qui est 11 en décimal.

En conclusion, avec plusieurs cas de tests, nous pensions que notre module racine_carree.vhd marche bien avec différents cas, alors nous passons donc à le test bench pour simuler tous les cas exhaustifs en faisant la vérification qu'on est pas plus que plus ou moins de la valeur exacte de la racine carrée.

Partie 2 : Banc d'essai

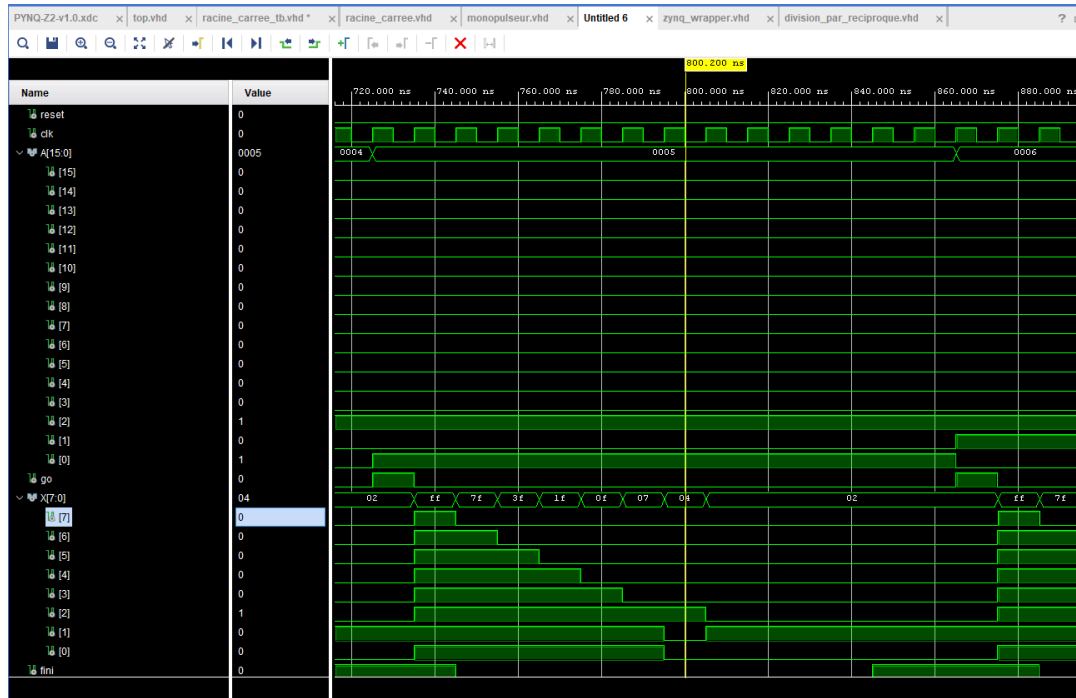


Figure 4. simulation1 du banc d'essai du module `racine_carree_tb.vhd`

Comme vous pouvez le constater à la figure 4, on a simulé un cas précis ici où le A est entrée et prend la valeur de 4 et on voit qu'à chaque cycle d'horloge, une itération est effectuée, ce qui engendre un changement sur la valeur de X comme sortie et après 10 itérations. On obtient bien à la fin de ces itérations que la racine de 4 est bien et bel de 2 comme attendu.

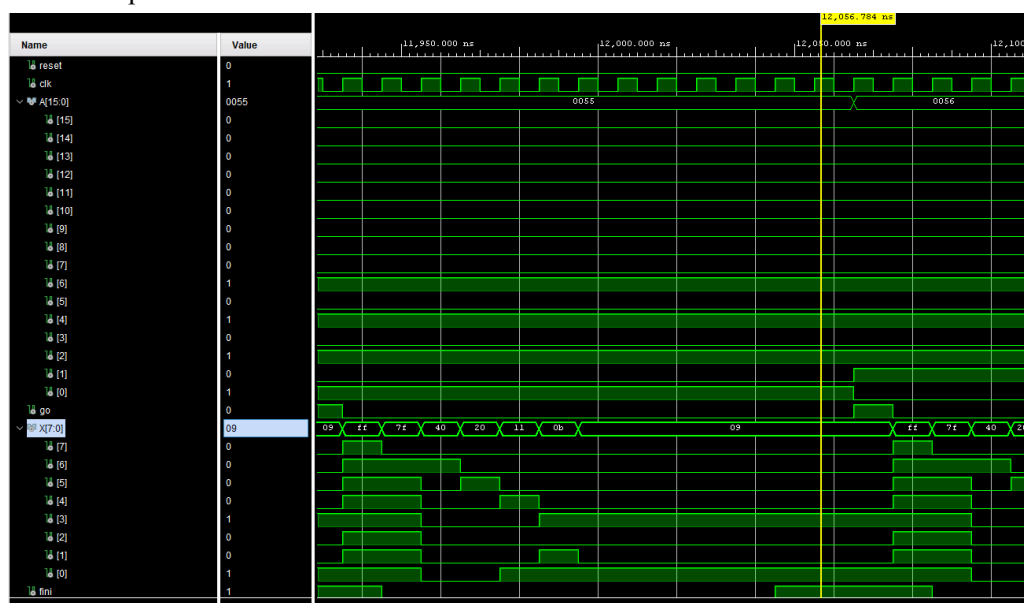


Figure 5. simulation2 du banc d'essai du module `racine_carree_tb.vhd`

Comme vous pouvez le constater on a simulé à la figure 5 un autre cas précis où on teste comme valeur de A de 0x55 ce qui est 85 en décimal, cela nous donne une racine carré de 9 comme attendu puisque la valeur réelle de la racine carrée est de 9,21954457 ce qui donne une erreur de $0.21954457 < 1$, ce qui est la condition à respectée.



Figure 6. simulation3 du banc d'essai du module `racine_carree_tb.vhd`

Comme vous pouvez le voir nous avons simulé aussi un cas limit où A prend une valeur plus grande que 0x80, on voit que cela cause un erreur de div par 0 dans le testbench car la valeur est trop grande et overflow lorsqu'on fait la première itération:

```
- quotient <= to_ufixed(std_logic_vector(num *
ROM_reciproque(to_integer(denom))), W_num - 1, -W_frac);
-- quotient <= num * ROM_reciproque(to_integer(denom));
```

On dépasse les valeurs que quotient peut prendre, ce qui explique pourquoi des bits sont coupés et on passe de ff à 3 pour la valeur de x avec seulement la première itération.

Donc, en conclusion, nous avons simulé toutes les valeurs possibles du test bench de A de 0 à 65535, pour chacune d'entre elles, on voit chacune des 10 itérations après que le signal go est activé et lorsque fini prend la valeur de 1, on passe à la prochaine valeur de A. On a aussi trouvé de la sortie X est toujours plus petite ou inférieure à 1, ce qui est attendu. Par contre, on remarque qu'il y a un problème avec le module `division_par_reciproque` puisque quotient est un `ufixed(15 downto -14)` et alors quand on fait `quotient <= num * ROM_reciproque(to_integer(denom));` num est aussi 15 downto 0 et si cette valeur est assez élevée, lorsqu'on fait le produit avec la `ROM_reciproque`, il produit un débordement. Il serait pertinent de peut-être optimiser cela pour avoir la chance de visualiser sur la simulation les valeurs dépassant 0x80.

Partie 3 : Implémentation sur la carte

```

Entrée [54]: from pynq import MMIO

demo = MMIO(0x41200000, 0x10)

writeData = 1
demo.write(0x0, writeData)

Entrée [55]: resultat1 = 0
resultat1 = demo.read(0x8)
print(f"Racine carrée est : {resultat1}")

Racine carrée est : 1

Entrée [56]: writeData = 144
demo.write(0x0, writeData)

Entrée [57]: resultat2 = 0
resultat2 = demo.read(0x8)
print(f"Racine carrée est : {resultat2}")

Racine carrée est : 12

Entrée [58]: writeData = 32768
demo.write(0x0, writeData)

Entrée [59]: resultat3 = 0
resultat3 = demo.read(0x8)
print(f"Racine carrée est : {resultat3}")

Racine carrée est : 181

Entrée [60]: writeData = 48592
demo.write(0x0, writeData)

Entrée [61]: resultat4 = 0
resultat4 = demo.read(0x8)
print(f"Racine carrée est : {resultat4}")

Racine carrée est : 219

Entrée [62]: writeData = 65535
demo.write(0x0, writeData)

Entrée [63]: resultat5 = 0
resultat5 = demo.read(0x8)
print(f"Racine carrée est : {resultat5}")

Racine carrée est : 255

```

Figure 7. Jupiternotebook pour des tests

Afin de vérifier le bon fonctionnement du module `racine_carree`, plusieurs tests ont été réalisés à l'aide d'un script Python. Chaque test consiste à écrire une valeur 16 bits (`i_A`) dans le registre d'entrée du module, à lancer le calcul, puis à lire la valeur de sortie (`o_X`), correspondant à la racine carrée entière de l'entrée. Les valeurs de test ont été choisies de manière à couvrir l'ensemble de la plage d'entrée possible (0 à 65535), en incluant : la valeur minimale, la valeur maximale, la valeur médiane, ainsi que deux valeurs intermédiaires pour observer la continuité et la précision du calcul. Ces tests montrent que le module calcule correctement la racine carrée!

Partie 4 : Défi - au-delà du A

4a. Une meilleure estimation de X_0 :

Le if ... elsif améliore le choix initial X_0 en fonction de la magnitude de l'entrée A. Cela rend l'algorithme de Newton plus rapide, plus précis et plus robuste, même avec un nombre réduit d'itérations kmax!

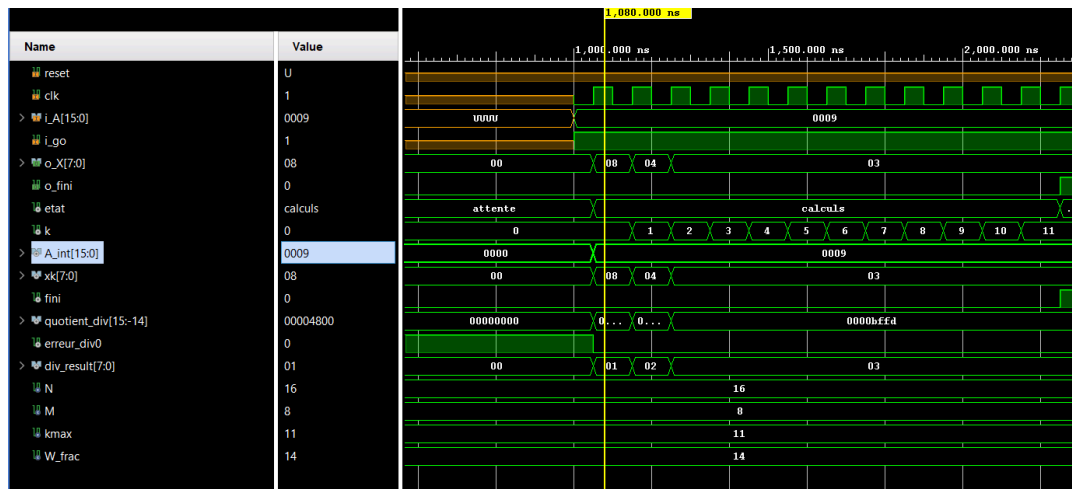


Figure 8. simulation du module `racine_carree.vhd` avec meilleure X_0

En seulement 2 itérations, on trouve la racine carré de 9 qui est 3.

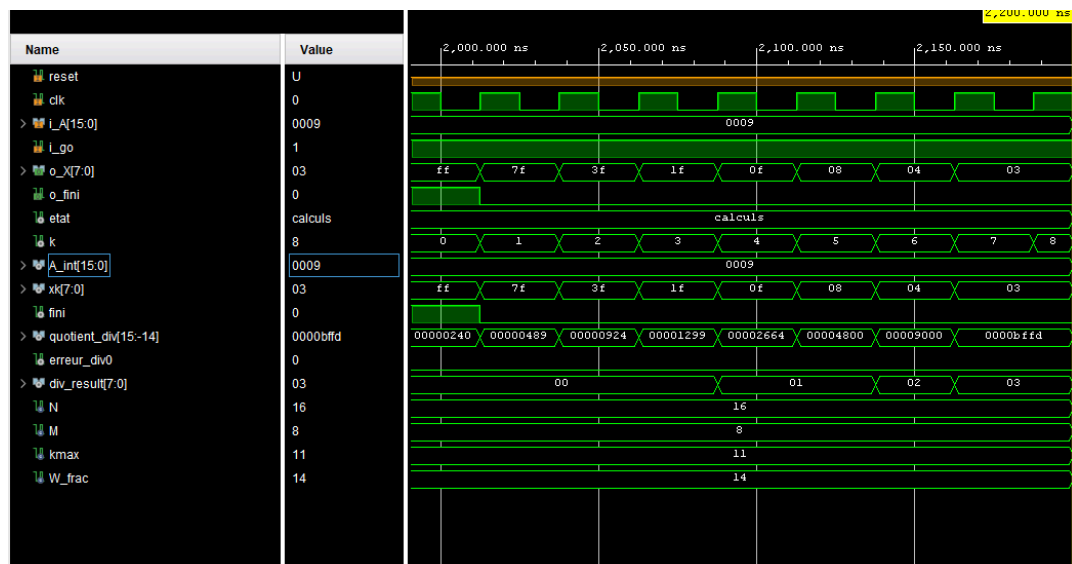


Figure 9. simulation du module `racine_carree.vhd` avec $X_0 = 255$

En 7 itérations, on trouve la racine carré de 9 qui est 3.

En conclusion, on voit une différence de 5 itérations entre la simulation avec des meilleures valeurs de X_0 , ce qui produit une grande différence en termes d'efficacité.